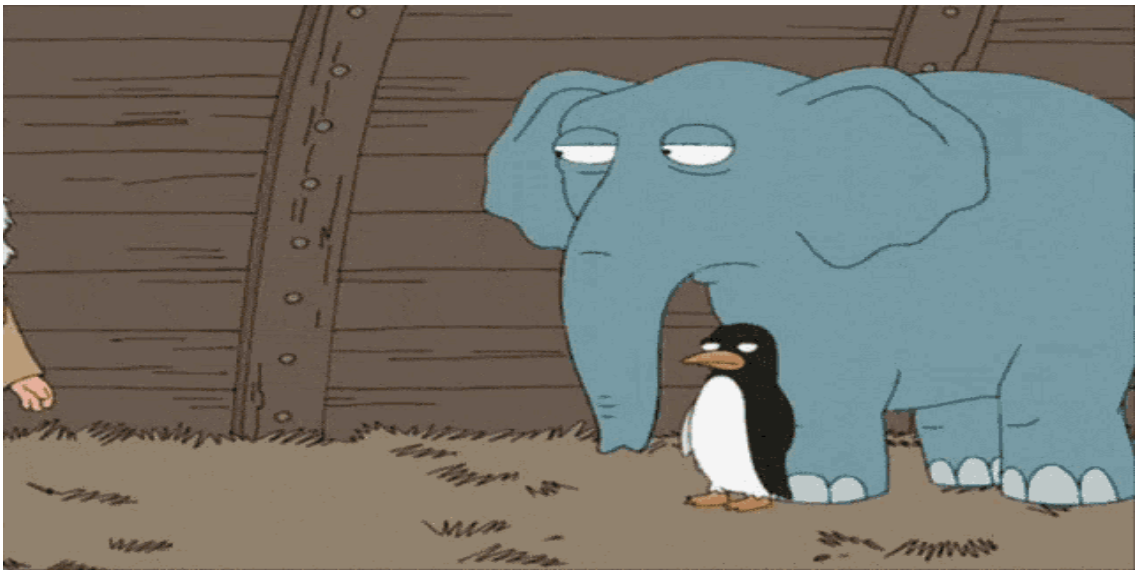
DealSim

This document describes the architecture and design behind the **AI-powered roleplay and evaluation platform**. It walks through the system's core components, how AI-driven conversations and real-time analytics are handled, and how structured evaluations are generated after each session. The document also captures key product insights, anticipated engineering challenges, and the design choices made to build a scalable and effective training platform.

This system design document represents my architectural approach to solving the problem. The design focuses on building a scalable AI-powered roleplay system capable of generating realistic sales conversations and structured performance evaluations.This **System Design Document is entirely my own work**. Which is probably exactly why there's plenty of room to refine and improve it.



USER JOURNEY

Step 1 – User Access
The user accesses the platform through the frontend, and requests are routed via the **Load Balancer / API Gateway**.

Step 2 – Dashboard Load
The request reaches the **User Dashboard Service**, which loads assigned simulations and past reports.

Step 3 – Scenario Creation (Admin)
Admins or managers create simulation scenarios by defining buyer personas, context, and evaluation metrics.

Step 4 – Training Assignment
Managers assign specific simulations to users as part of their training.

Step 5 – View Past Reports
Users can review previously completed simulations and evaluation reports.

Step 6 – Start Simulation
When the user starts a simulation, the request is routed to the **Simulation Service**.

Step 7 – Simulation Initialization
The Simulation Service invokes the **AI Roleplay & Evaluation Engine** to start the roleplay session.

Step 8 – Live Conversation
The user interacts with the AI buyer persona through the frontend simulation interface.

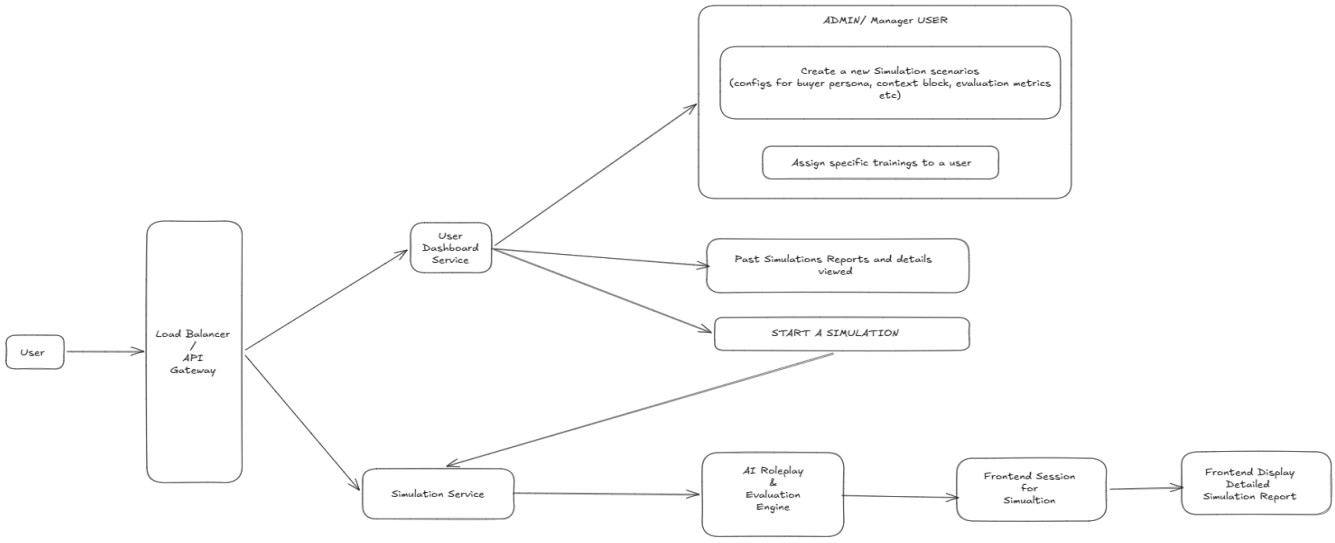
Step 9 – Session Evaluation
After the session ends, the engine processes the transcript and generates structured evaluation metrics.

Step 10 – Report Display
The evaluation results are returned to the frontend and displayed as a detailed simulation report.

HIGH LEVEL ARCHITECTURE

Let's start with an overall High Level Architecture.
Below are the major components of the system. This is a 10,000 feet overview. The real effort is below in the AI Engine described below.

1. **Load Balancer / API Gateway -** Routes incoming requests and distributes traffic to backend services.
2. **User Dashboard Service-** Manages dashboard data, assigned simulations, and historical reports.
3. **Admin / Manager Module**
   Allows admins to create simulation scenarios and configure personas, context, and evaluation metrics.
4. **Training Assignment Module -** Assigns specific training simulations to users.
5. **Simulation Service -** Orchestrates simulation sessions and coordinates with the AI engine.
6. **AI Roleplay & Evaluation Engine -** Generates AI buyer responses, tracks conversation state, and evaluates user performance.
7. **Frontend Simulation Session -** Provides the chat interface where users conduct roleplay simulations.
8. **Simulation Report Display -** Displays detailed evaluation results and insights from completed sessions.
   ARCHITECTURE DIAGRAM -
   https://drive.google.com/file/d/1NXLOSGrJeWuU_A11bQg1vEK57AvHMGmF/view?usp=sharing

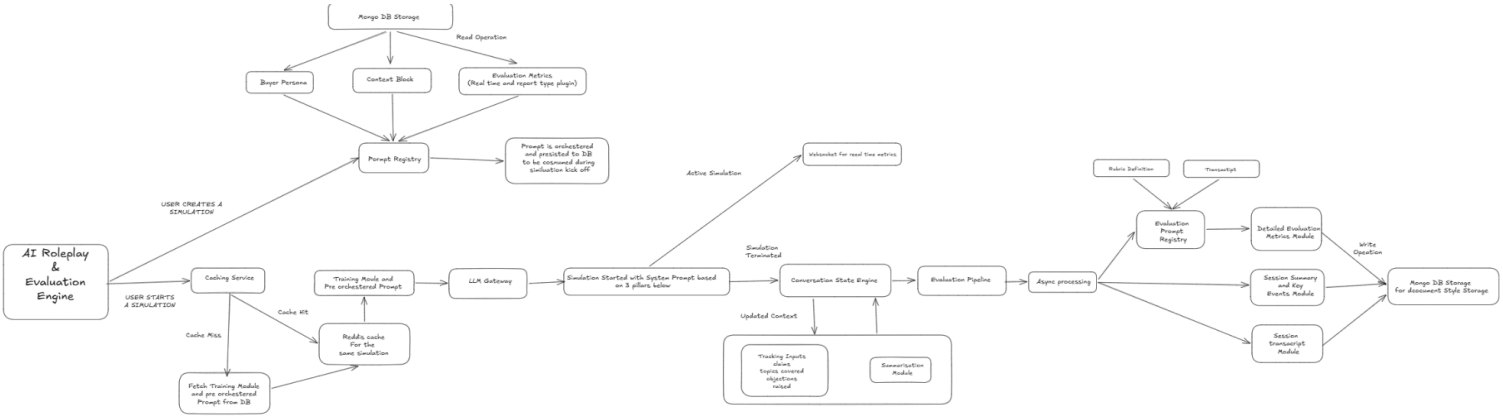


AI Roleplay & Evaluation Engine

We plan to build a microservice **ai_roleplay_engine_ms** which will serve as the core simulation and evaluation engine for AI-driven sales training sessions. Its primary responsibility is to orchestrate realistic buyer simulations, track conversation context, generate real-time analytics, and produce structured post-session evaluations. The following modules are suggested within the service:

1. **Prompt Registry Module:**
   This module is responsible for managing the prompts used to initiate and control AI simulations. It combines buyer persona definitions, scenario context, and evaluation rubrics to generate structured prompts for the AI conversation engine.
2. **Simulation Orchestrator Module:**
   This module is responsible for initiating and managing AI roleplay sessions. It prepares the system prompts, injects persona and scenario context, and coordinates interactions between the user and the AI buyer during a session.
3. **Conversation State Engine:**
   This module is responsible for maintaining the evolving state of the conversation. It tracks key claims made by the seller, topics discussed, objections raised by the buyer, and maintains contextual memory for more realistic AI responses.
4. **LLM Gateway:**
   This module acts as an abstraction layer for interacting with external language models. It handles prompt formatting, response parsing, retry logic, and ensures consistent communication with the AI model provider.
5. **Analytics Engine:**
   This module is responsible for computing real-time conversation metrics such as words-per-minute, filler word frequency, talk ratio, and monologue detection. These metrics are streamed to the frontend via WebSockets during the live session.
6. **Summarization Module:**
   This module manages context window limitations by summarizing earlier parts of the conversation when necessary, ensuring that the AI model receives a concise but meaningful context for generating responses.
7. **Evaluation Pipeline:**
   This module is responsible for processing completed sessions and generating structured evaluations. It uses the full conversation transcript and evaluation rubric to produce competency scores and qualitative feedback.

8. **Evaluation Metrics Module:**
   This module processes and stores detailed evaluation results, including per-competency scores, feedback with transcript evidence, and an aggregated performance score for the session.
9. **Session Transcript Module:**
   This module stores the entire conversation history between the user and the AI buyer, ensuring that transcripts are preserved for evaluation, analysis, and future reference.
10. **Session Summary & Events Module:**
    This module generates a structured summary of the session including key moments, objections raised, and major conversation milestones to help users quickly understand their performance.
11. **Caching Module (Redis):**
    This module caches frequently used simulation configurations such as training modules and prompt structures to reduce latency and improve session initialization performance.



ARCHITECTURE DIAGRAM -

https://drive.google.com/file/d/1NXLOSGrJeWuU_A11bQg1vEK57AvHMGmF/view?usp=sharing

Data Model

The platform stores structured information related to organizations, users, training simulations, roleplay sessions, and evaluation results. Since the system deals heavily with **conversation transcripts, analytics metrics, and evolving evaluation structures**, **MongoDB** is used as the primary datastore.

Why MongoDB

MongoDB fits this use case because simulation sessions contain nested, semi-structured data such as transcripts, analytics snapshots, conversation state, and summaries. Storing these artifacts within a single document simplifies retrieval, avoids join-heavy queries, and allows flexible schema evolution for rubrics and evaluations.
The system is organized into the following MongoDB collections.

1. **Organization -** Represents a company or team using the training platform.
   Fields:
   - _id
   - name
   - industry
   - createdAt

   Each organization can have multiple users, simulations, and training sessions.

2. **User -** Represents a user within an organization.
   Fields:
   - _id
   - organizationId
   - name
   - email
   - role (organization_admin | org_employee)
   - createdAt
   - Roles:
     - **organization_admin** – Creates simulations and assigns training.

    - **org_employee** – Participates in simulations and views reports.

3. **Persona -** Represents the simulated buyer persona used during roleplay sessions.
   Fields:

   - _id
   - organizationId
   - name
   - role
   - company
   - personalityTraits
   - resistanceLevel
   - defaultObjections[]
   - createdAt

   Personas define the behavioral characteristics of the AI buyer during the simulation.

4. **Context -** Represents the sales situation used in a simulation.
   Fields:

   - _id
   - organizationId
   - product
   - dealSize
   - salesStage
   - specialConditions
   - createdAt

   Context provides the business background that guides AI responses.

5. **Rubric -** Defines how simulation performance will be evaluated.
   Fields:

   - _id
   - organizationId
   - name
   - competencies[]
   - createdAt
   - Each competency contains:
   - name
   - description

- weight
- scoringGuidelines

Rubrics allow organizations to evaluate sessions using different training frameworks.

6. **Simulation -** A simulation combines persona, context, and rubric to create a reusable training scenario. When a simulation is created, the Prompt Registry generates an orchestrated system prompt that combines persona instructions, context details, and evaluation guidelines. This prompt is stored with the simulation so that sessions can start instantly without recomputing prompt structure.

   Fields:

   - _id
   - organizationId
   - name
   - personaId
   - contextId
   - rubricId
   - orchestratedPrompt
   - createdAt

   Instead of a single string prompt, we store **structured prompt components**.
   Example:
   "orchestratedPrompt": {
   "systemPrompt": "...",
   "personaPrompt": "...",
   "contextPrompt": "...",
   "evaluationInstructions": "..."
   }

   This design enables reusable components to be combined into multiple simulations.

7. **Session -** Represents a single roleplay interaction between a user and the AI buyer. Session data is stored as a **MongoDB document containing transcripts, analytics metrics, and conversation state**.

   Fields:

   - _id
   - simulationId
   - userId
   - status (in_progress | completed | evaluated)

- transcripts[]
- analyticsSnapshots[]
- conversationState
- summary
- overallSummary
- keyEvents[]
- evaluationId
- createdAt

8. **Transcripts -** Transcripts represent the conversation history between the seller (user) and the AI buyer. Each transcript entry represents a conversational turn.

   Fields:

   - speaker (seller | buyer)
   - content
   - wordCount
   - timestamp

   These transcripts are used by the conversation engine, analytics system, and evaluation pipeline.

9. **Analytics Snapshots -** Stores analytics metrics generated during the simulation session.

   Fields:

   - wpm
   - fillerWordCount
   - talkRatio
   - monologueFlag
   - timestamp

   These metrics enable real-time feedback during the roleplay session.

10. **Session Summary & Key Events -** After the session ends, the system generates a structured summary highlighting important moments in the conversation.

    Fields:

    - overallSummary
    - keyEvents[]

Key events may include discovery questions, objections raised, or value propositions discussed.

11. **Evaluation -** Stores structured evaluation results generated after the simulation completes.

    Fields:

    - _id
    - sessionId
    - competencyScores[]
    - overallScore
    - feedback
    - createdAt
    - Each competency score contains:
    - name
    - score
    - justification
    - transcriptEvidence

    Evaluation results are displayed in the simulation report dashboard.

API DESIGN

The backend exposes RESTful APIs that support the lifecycle of **organization setup, user management, simulation configuration, roleplay sessions, and evaluation retrieval**. These APIs are consumed by the frontend dashboard and orchestrate interactions with the AI Roleplay & Evaluation Engine.

1. **Organization APIs -** These APIs manage organization-level configuration.

   Create Organization
   POST /organizations - Creates a new organization account.
   Get Organization
   GET /organizations/{organizationId} - Returns organization details.

2. **User APIs -** These APIs manage users within an organization.

   Register User
   POST /users - Creates a new user in the organization.
   User Login

POST /auth/login - Authenticates a user and returns a session token.

Get User Profile

GET /users/{userId} - Returns details for a specific user.

3. **Persona APIs -** These APIs allow administrators to manage reusable buyer personas.

   Create Persona

   POST /personas - Creates a new buyer persona configuration.

   List Personas

   GET /personas - Returns all personas for the organization.

4. **Context APIs -** These APIs define reusable sales contexts.

   Create Context

   POST /contexts - Creates a new simulation context.

   List Contexts

   GET /contexts - Returns all contexts available to the organization.

5. **Rubric APIs -** These APIs define evaluation frameworks used in simulations.

   Create Rubric

   POST /rubrics - Creates a rubric containing evaluation competencies.

   List Rubrics

   GET /rubrics - Returns available rubrics.

6. **Simulation APIs -** Simulations combine persona, context, and rubric into reusable training scenarios.

   Create Simulation

   POST /simulations

   Creates a new simulation and generates the orchestrated system prompt.

   List Simulations

   GET /simulations - Returns simulations available to the organization.

   Get Simulation

   GET /simulations/{simulationId} - Returns detailed simulation configuration.

7. **Session APIs -** These APIs manage the lifecycle of roleplay sessions.

   Start Session

   POST /sessions - Creates a new session and initializes conversation context using the simulation's orchestrated prompt.

   Send Message

POST /sessions/{sessionId}/message - Processes the seller's message, generates the AI buyer response, updates transcripts, and computes analytics. This endpoint also triggers analytics computation and conversation state updates.

End Session
POST /sessions/{sessionId}/end - Marks the session as completed and triggers the evaluation pipeline.

8. **Evaluation APIs -** These APIs expose structured evaluation results after a session completes.

Get Evaluation

GET /sessions/{sessionId}/evaluation - Returns competency scores, feedback, and transcript evidence.

9. **WebSocket Events -** During an active simulation session, real-time analytics are streamed to the frontend using WebSockets.

Subscribe to Session

Client → Server

```
{
"type": "subscribe",
"sessionId": "session_123"
}
```

Analytics Update

Server → Client

```
{
"type": "analytics_update",
"data": {
  "wpm": 145,
  "fillerWords": 3,
  "talkRatio": 0.62,
  "monologueFlag": false
}
}
```

Session Ended

Server → Client

```
{
"type": "session_ended",
"data": {
  "evaluationId": "eval_456"
}
}
```

End to End Session Flow

This section describes how a simulation session is executed from initialization to evaluation.

1. Session Initialization

A user selects a simulation and starts a session.

POST /sessions

The **Simulation Service** retrieves the simulation configuration and its **orchestrated prompt** from MongoDB. A new session document is created with:

status: in_progress
transcripts: []
analyticsSnapshots: []
conversationState: {}

The session ID is returned to the frontend and a WebSocket connection is established for real-time analytics.

2. User Sends Message

When the user sends a message:

POST /sessions/{sessionId}/message

The message is appended to the session's **transcripts array**.

Example:

```
{
speaker: "seller",
content: "...",
wordCount: 12,
timestamp: "..."
}
```

3. Conversation Engine Processing

The **AI Roleplay Engine** constructs the LLM prompt using:

```
systemPrompt
personaPrompt
contextPrompt
conversationState
recentTranscripts
```

The request is forwarded to the **LLM Gateway**, which generates the AI buyer response.

The response is appended to the session transcript.

4. State & Analytics Update

The **Conversation State Engine** updates structured state:

```
claimsMade
topicsCovered
objectionsRaised
objectionsResolved
```

The **Analytics Engine** computes metrics:

```
wpm
fillerWordCount
talkRatio
monologueFlag
```

Analytics snapshots are stored and pushed to the frontend via WebSocket.

5. Session Completion

When the user ends the session:

POST /sessions/{sessionId}/end

The session status is updated to **completed**, and the **Evaluation Pipeline** is triggered.

6. Evaluation & Summary

The evaluation engine processes:

session transcripts
simulation rubric
conversation summary
structured evaluation prompt

It generates:

- competency scores
- transcript evidence
- feedback
- overall score

The system also generates a **session summary and key events**, which are stored with the session.

7. Results Display

The frontend retrieves results:

GET /sessions/{sessionId}/evaluation

The simulation report displays evaluation scores, feedback, transcripts, and session analytics.

Design Decisions and Trade-offs

MongoDB for Session Storage
MongoDB was chosen because session data contains nested structures such as transcripts, analytics snapshots, and conversation state. Embedding these artifacts in a single document simplifies retrieval and avoids join-heavy queries.

Prompt Pre-Orchestration at Simulation Creation
The orchestrated prompt is generated when a simulation is created rather than at session start. This reduces runtime overhead and allows sessions to initialize instantly.

Embedded Transcripts in Session Documents
Transcripts are stored within the session document instead of a separate collection. Since sessions are typically retrieved as a whole, this improves read performance and simplifies data access.

Separate Evaluation Pipeline
Evaluation is triggered after the session ends instead of during the conversation. This prevents evaluation latency from impacting real-time interaction.

Conversation State Tracking
Structured conversation state (claims, topics, objections) is maintained alongside transcripts. This adds complexity but enables more realistic AI buyer behavior and consistent responses.

Future Improvements

Agent Initialization Caching
Currently, a new AI agent context is initialized when a session starts. In the future, we could introduce a caching layer for pre-initialized agents based on simulation configuration. This would allow multiple users to reuse the same initialized prompt context, reducing startup latency and LLM overhead.

User-Created Practice Simulations
Employees could be allowed to create their own custom training simulations for self-practice by defining persona, context, and evaluation criteria. Since each simulation incurs LLM costs, this feature would be rate-limited to control resource usage.

Advanced Analytics
Extend analytics beyond WPM and filler words to include conversation sentiment, objection handling effectiveness, and discovery depth.